

Machine Learning: Project (2025-2026)

Multi-Agent Learning in Canonical Games and Knights Archers Zombies

Giuseppe Marra, Wannes Meert

February 2026

1 Introduction and Related Literature

We live in a visual, multi-agent world and to be successful in that world, agents need to learn to recognize what they see and take into account the agency of others. They will need to communicate with others and coordinate their plans. Examples include self-driving cars interacting in traffic, personal assistants acting on behalf of humans, and robotic teams.

This assignment covers topics in object recognition and multi-agent reinforcement learning (MARL). We assume elementary knowledge of vision models and single-agent reinforcement learning.¹ When moving from single-agent RL to multi-agent RL, Game Theory plays an important role as it is a theory of interactive decision making. Throughout the assignment you will use some elementary game theoretic concepts in combination with multi-agent learning, which is non-stationary and reflects a moving target problem.²

In this assignment we first tackle some canonical games from the ‘pre-Deep Learning’ period. To learn how (multi-agent) reinforcement learning and game theory relate to each other, you will work with tabular RL methods using ϵ -greedy and Boltzmann exploration [9, 2] and interpret the evolution of the learned policy. Next, we move to the Knights Archers and Zombies game using RL and ML [4, 3].

We will be working in the PettingZoo environment³ and recommend the RLLib⁴ framework for the RL algorithms. Learning how to use advanced, state-of-the-art software toolboxes for AI is part of the project and we expect you to explore the documentation (including manuals, docstrings, code examples, etc.). We expect knowledge about Python.

2 Approach

You work on this project in a team of 1 or 2 students. Questions about any part can be directed to any of the team members. The goal of this project is for the students to obtain hands-on experience with machine learning, and to deepen their insight in some of the topics taught in the machine learning course. The evaluation of the project aims to assess to what extent this goal is reached, for each individual student.

¹ A good reference when less familiar with vision models is: <https://huggingface.co/learn/computer-vision-course/>

A good reference when less familiar with RL is: <http://incompleteideas.net/book/the-book-2nd.html>

² see [6] or [5] for basic concepts about game theory when less familiar

³ https://pettingzoo.farama.org/environments/butterfly/knights_archers_zombies/

⁴ <https://docs.ray.io/en/latest/rllib/index.html>

We expect that you have all code available on the computers at the Department of Computer Science, that your code runs in that environment, and that you participate in the tournament. The report is submitted via Toledo.

The authorship of each piece of the source code and the report must be clear and unambiguous. If parts of the code have been taken from elsewhere (e.g., copied from the internet), this must be indicated very clearly in the code. The report must provide a clear view on what has been copied from elsewhere, and what is your own work. The report itself must adhere to general scientific standards of source attribution.

Note that since a lot of code is available within the available frameworks and AI tools we do expect students to show a good understanding of the techniques deployed, and be able to conduct a knowledgeable conversation about the techniques mentioned in the report. You are also expected to be able to explain all submitted code.

We use the Dept CS Assignment Commons ES-GW-TP-TS-NPP-NVP.⁵ Summarized: All resources are allowed but code or text that is claimed to be authored by the team and cannot be explained or reappears in other submissions/sources is assumed to be copied. This means that the part in question may be dropped from grading and may be cause for sanctions.

Please direct questions that you have about the project to the Toledo forum or the classroom discussion moments such that all students can benefit from the discussion or participate to offer answers. You are also allowed to ask technical questions about the tools mentioned in this assignment. Questions about other tools are allowed, but there is no guarantee that they can be answered (e.g., PyCharm, VS Code).

3 Deadlines

3.1 Form Groups

Before February 27th, 23:59

Mail the team member names to wannes.meert@kuleuven.be and giuseppe.marra@kuleuven.be.

3.2 Submit Draft of Report (optional, not graded)

Before March 20th, 23:59

If you submit a draft report via Toledo, feedback will be provided individually.

3.3 Upload all code and submit agent to tournament

Before May 15th, 23:59

Upload all code for all tasks. For the final evaluation of your agents for Tasks 3 and 4 we will play a tournament, in which each agent will play many games. The tournament is played with all submitted agents and a range of simple baseline agents. This will be used to assess whether your agent learned how to play the game.

To participate in the tournament, follow the predetermined template and upload your agent to the departmental computers. See <https://github.com/ML-KULeuven/ml-project-2025-2026> for technical instructions. If you work in a team, choose the directory of one member. Test your (preliminary) code as early as possible on the departmental computers. An implementation that does not run reduces your score.

⁵<https://wms.cs.kuleuven.be/cs/english/study/assignment-commons>

3.4 Submit Report

Before May 15th, 23:59

Submit your report (PDF, ≤ 10 pages, excluding references) to Toledo. Your report should fulfill the following criteria:

- Mention the directory on the dept. computers where your **code for all tasks** and agents are stored.
- Formulate your **design choices** as research questions and answer them.
- Write out the (scientifically supported) **conclusions** you draw from your experiments.
- Be concrete and precise about methods, formulas and numbers. A scientific text is **reproducible**.
- Clearly **cite** sources.
- Report, per person, the **time each of you spent** on the project, and how it was divided over the tasks.
- An appendix is allowed for additional results or figures you want to refer to during the discussion (pages > 10). There is no guarantee the appendix is considered and the first 10 pages need to be fully self-contained.

3.5 Peer assessment

Before May 15th, 23:59, individually

Send by email a peer assessment of your partner's efforts. This should be done on a scale from 0-4 where 0 means "My partner did not contribute", 2 means "I and my partner did about the same effort", and 4 means "My partner did all the work". Add a short motivation to clarify your score. This information is used only by the professors and assistants and is not communicated further.

3.6 Oral discussion

Week of May 18th

Discussion about your report and code. Slots will be available on Toledo.

4 Tasks

Your report should discuss the following tasks (mention the task numbers).

[The final mark per task is determined by the combination of the report, the code and the oral discussion]

Task 1: Literature Study

Describe the 3 papers that influenced your approach the most, and explain why. You are expected to at least read the relevant sections in the provided references to understand the terminology used in this assignment.

[With this task you can earn $\frac{1}{20}$ points of your overall mark.]

Task 2: Learning & Dynamics: Matrix Games

Here we learn how to play four benchmark matrix games: Stag Hunt, Subsidy Game, Matching Pennies and Prisoner's Dilemma. Use the payoff tables in Figure 1. These games belong to different categories of games, i.e. social dilemma, zero-sum or coordination game.

Goal

You train a policy with basic RL algorithms for both players per benchmark matrix game using independent learning. Both players use the same RL algorithm. You can use self-play (agents use the same model).

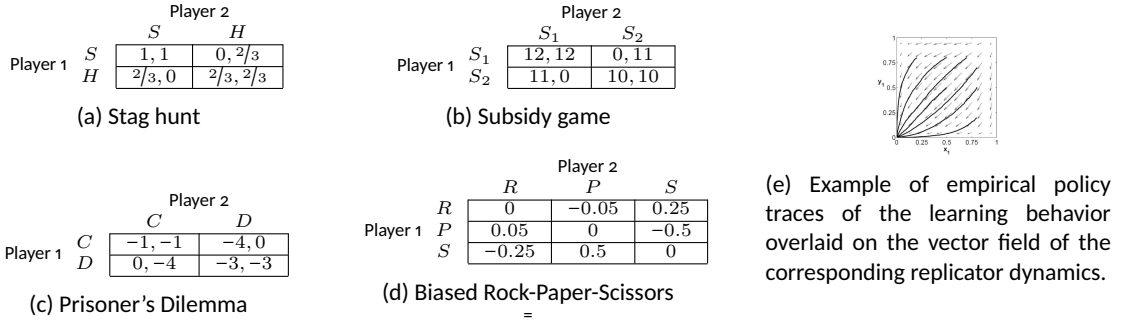


Figure 1: Matrix games

1. List for each game the Nash equilibria and Pareto Optimal states. [$\frac{1}{8}$ points]
2. Implement yourself (a) ϵ -greedy Q-learning, (b) Boltzmann Q-learning, and (c) Lenient Boltzmann Q-learning [9, 1]. Plot multiple empirical (time-averaged) learning trajectories. Thus, show how the policy changes over multiple iterations of the learning step (see figure 1e for an example). Explain the behavior and the differences between algorithms. Investigate and report on whether the learning algorithms converge to a Nash equilibrium and/or a Pareto optimal state (or why not). [$\frac{5}{8}$ points]
3. For matrix games, we can analytically verify whether your learning trajectories are behaving as expected. You can do this by computing the expected equilibrium and by computing the replicator equations and plotting the directional (vector) field plots [1]. Do this for Boltzmann Q-learning and Lenient Boltzmann Q-learning. You can compute the equations yourself and make quiver plot or use a library like OpenSpiel.⁶ [$\frac{1}{8}$ points]

[With this task you can earn $\frac{7}{20}$ points of your overall mark.]

Task 3: Playing the Knights-Archers-Zombies game

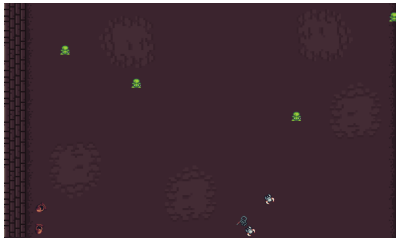
In this task you will train an agent to control two archer agents in the Knights-Archers-Zombies game (KAZ, Figure 2a) in the PettingZoo environment⁷. In this game, agents need to hit as many zombies as possible before either they get hit or a zombie reaches the bottom line. We will use the *pixel-based visual observations*. Additionally, you need to use the environment provided in the template where distortions have been added to have an increasingly difficult vision task.

You will employ Reinforcement Learning (RL) techniques [7, 8] to develop your solution. In simple matrix games, learning action probabilities (i.e. policies or strategies) directly is feasible because matrix games are stateless, synchronous, single-step interaction games. In fact, in a matrix game, players choose their actions simultaneously, get the corresponding reward and the game resolves in a single step. This is not the case anymore in KAZ, where current actions influence not only immediate rewards but also future states (i.e., a Markov Decision Process). For instance, if only one zombie is present, the best move might be to shoot immediately. Conversely, facing multiple zombies may require repositioning before attacking.

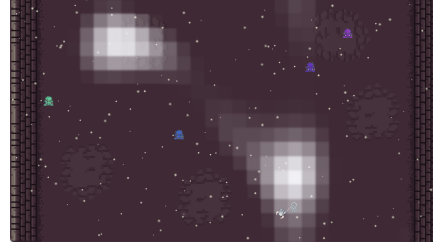
Due to the vast number of state-action combinations in KAZ, standard model-free approaches using tabular representations are impractical. You cannot just store the probabilities (or values) of all the actions for all the

⁶<https://openspiel.readthedocs.io/en/latest/> (leniency is not supported out of the box)

⁷https://pettingzoo.farama.org/environments/butterfly/knights_archers_zombies/



(a) KAZ environment without distortions.



(b) KAZ environment with distortions.

Figure 2: Knights-Archers-Zombies (KAZ) environment.

possible states. Instead, generalization techniques are necessary, where the information learned in one state can be transferred and re-used in other states. One common approach is leveraging deep neural networks to predict the value or action distribution of states. This requires designing and/or learning features that effectively describe states and actions, allowing the model to generalize well.

Feature representation strategies include:

- Manual feature engineering: Preprocessing states to extract or compute features that simplify learning.
- Automated feature learning: using raw data as input to a deep learning model. You should think about the correct architecture and how this can be trained.
- Hybrid approach: Combining manual and automated feature extraction.

You may leverage implementations from PettingZoo⁸ and Ray RLlib⁹. Machine learning models, such as deep neural networks, can represent state-action value functions (Q -values), state value functions (V -values), or directly learn policies. You are free to choose any RL technique, such as deep Q-learning, policy gradient methods, or Proximal Policy Optimization (PPO)—many of which are available in RLlib.

The vision part (i.e. to recognize the zombies) is increasingly difficult with distortions that are added. Provide a vision model that can deal with as many distortions as possible. There are six levels: (0) no distortion, (1) stars, (2) clouds, (3) different colors, (4) distorted pixel in zombies, and (5) waves over the entire screen. In the evaluation, the distortions might be combined differently.

As this is a multi-agent setting, you will have multiple choices on how to design and/or train your agents (e.g. duplicate the same agent, train two different agents, etc.). RLlib provides different multi-agent settings.

Goal

Your objectives for this task are (both are required to earn points):

1. **a. Implementation & Evaluation:** You will develop and train an agent for the two archers KAZ environment using the provided template. This implementation will use PettingZoo for environment interaction and state processing. You may choose a machine learning library to develop you agents; we provide code examples and recommend the RLlib library. You are expected to evaluate on your machine your agents' performance and include such evaluations in your report. You compare your

⁸<https://pettingzoo.farama.org/index.html>

⁹<https://docs.ray.io/en/latest/rllib/index.html>

agent against simple baselines you come up with (e.g., random play, always shooting diagonally, etc.) for the different levels of distortions.

b. Central Evaluation: Tournament: You upload the agent you trained, and the training code, to the departmental computers. Your agent will play a number of randomly initialized games. The random seed used for the evaluation is not disclosed in advance, neither is the number of zombies or the types of distortions. Your agent must handle any possible game (i.e. any possible zombie appearance, number and configuration). The average reward is computed and used to rank your agent with respect to baseline agents and agents implemented by other students. You are expected to beat the (unseen) baselines.

[a + b: 7/12 points]

2. **Central Evaluation: Zombie Detection:** Using the same code as in the previous step, a number of observation vectors with varying levels of distortion are given to your agent. The agent replies with the bounding boxes. The average precision is computed and used to rank your agent with respect to agents implemented by other students. You are expected to find at least 75% of the zombies without distortion. [5/12 points]

Use the results of your evaluation, along with relevant literature, to justify your design choices. Explicitly describe your model architecture (e.g., network structure, input tensor format), the observed gameplay behavior (e.g., what strategies does your model learn?) and the learning statistics you used to analyze performance.

Important: In submitting code for evaluation, you must not alter the original reward scheme provided by PettingZoo for the KAZ environment. However, you are allowed to modify the reward structure of the environment when training on your machine.

[With this task you can earn 12/20 points of your overall mark.]

References

- [1] Daan Bloembergen et al. "Evolutionary Dynamics of Multi-Agent Learning: A Survey". In: *J. Artif. Intell. Res. (JAIR)* 53 (2015), pp. 659–697.
- [2] Lucian Busoniu, Robert Babuska, and Bart De Schutter. "A Comprehensive Survey of Multiagent Reinforcement Learning". In: *IEEE Trans. Systems, Man, and Cybernetics, Part C* 38.2 (2008).
- [3] Ian J. Goodfellow, Yoshua Bengio, and Aaron C. Courville. *Deep Learning*. Adaptive computation and machine learning. MIT Press, 2016.
- [4] Yann LeCun, Yoshua Bengio, and Geoffrey E. Hinton. "Deep learning". In: *Nature* 521.7553 (2015), pp. 436–444.
- [5] Yoav Shoham and Kevin Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2009. URL: <http://www.masfoundations.org/mas.pdf>.
- [6] Lukas Schäfer Stefano V. Albrecht Filippas Christianos. *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. URL: <https://www.marl-book.com>.
- [7] Richard S. Sutton and Andrew G. Barto. *Reinforcement learning: An introduction*. 2nd. Cambridge, MA: MIT Press, 2017. URL: <http://incompleteideas.net/book/the-book-2nd.html>.
- [8] Csaba Szepesvári. *Algorithms for Reinforcement Learning*. Morgan & Claypool, 2010. URL: <https://sites.ualberta.ca/~szepesva/RLBook.html>.
- [9] Karl Tuyls and Gerhard Weiss. "Multiagent Learning: Basics, Challenges, and Prospects". In: *AI Magazine* 33.3 (2012), pp. 41–52.